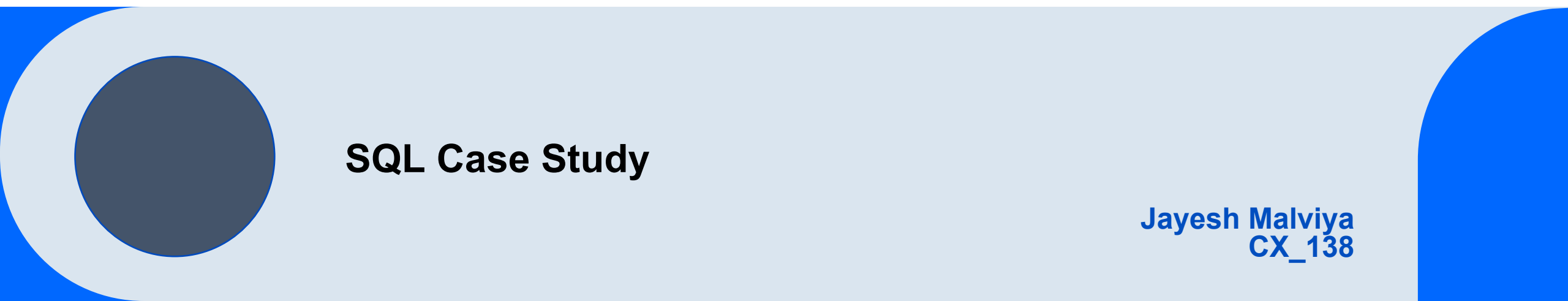




Customer Behavior Analysis in Online Food Delivery



SQL Case Study

Jayesh Malviya
CX_138

Introduction

Objective:

Analyze customer behavior using SQL

Focus:

- ✓ Revenue analysis
- ✓ Customer activity
- ✓ Gold membership impact

Dataset Overview

Tables

- ✓ Users
- ✓ Sales
- ✓ Product
- ✓ Goldusers_signup

Key columns

- ✓ user_id
- ✓ product_id
- ✓ Price
- ✓ created_date

Methodology & Approach

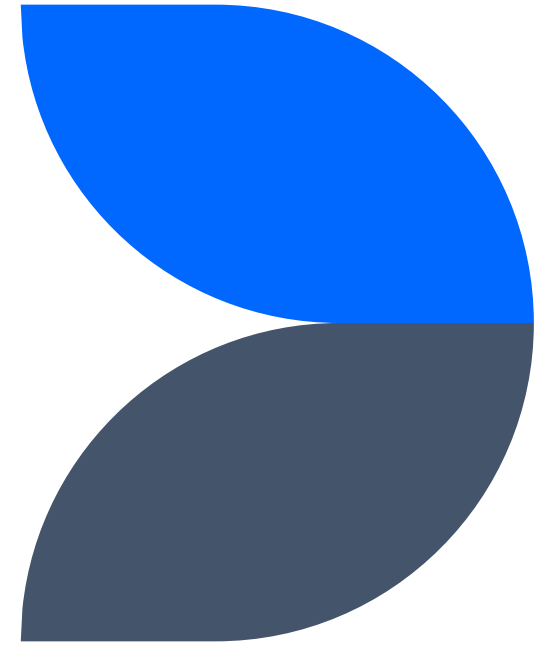
SQL Techniques Used:

- ✓ Joins
- ✓ Aggregations
- ✓ CTEs
- ✓ Window Functions

Approach:

Query → Analyze → Insight

SQL queries ->



Q1

```
1  -- Q1. What is the total sales revenue generated by each product?
2
3  • SELECT
4      p.product_id,
5      p.product_name,
6      SUM(p.price) AS total_revenue
7  FROM sales s
8  JOIN product p
9  ON s.product_id = p.product_id
10 GROUP BY p.product_id, p.product_name
11 ORDER BY total_revenue DESC;
```

Q2

```
13  -- Q2. Which 3 product has the highest sales revenue?
14
15 • SELECT
16     p.product_id,
17     p.product_name,
18     SUM(p.price) AS total_revenue
19 FROM sales s
20 LEFT JOIN product p
21 ON s.product_id = p.product_id
22 GROUP BY p.product_id, p.product_name
23 ORDER BY total_revenue DESC
24 LIMIT 3;
```

Q3

```
26  -- Q3. How many users have signed up for the service and has taken the gold membership?
27
28 • SELECT COUNT(*) AS users
29   FROM users;
30
31 • SELECT COUNT(*) AS gold_users_count
32   FROM goldusers_signup;
```


Q4

```
34  -- Q4. What is the revenue generated from gold users?
35
36 • SELECT
37     gu.user_id, sum(price) AS total_revenue
38 FROM goldusers_signup gu
39 LEFT JOIN sales s
40 ON gu.user_id = s.user_id
41 LEFT JOIN product p
42 ON p.product_id = s.product_id
43 GROUP BY gu.user_id
44 ORDER BY total_revenue DESC;
```

Q5

```
46  -- Q5. What is the total revenue generated from gold users?
47
48 • SELECT
49     sum(price) AS total_revenue
50 FROM goldusers_signup gu
51 LEFT JOIN sales s
52 ON gu.user_id = s.user_id
53 LEFT JOIN product p
54 ON p.product_id = s.product_id
55 ORDER BY total_revenue DESC;
```

Q6

```
57  -- Q6. Which users has been a gold user for the How much of time?
58
59 • SELECT
60     goldusers_signup.user_id,
61     user_name.user_name,
62     gold_signup_date,
63     datediff(curdate(), gold_signup_date) AS days_as_gold_members
64 FROM goldusers_signup LEFT JOIN user_name
65 ON user_name.user_id = goldusers_signup.user_id;
```

Q7

```
67  -- Q7. What is the most popular product among gold users?
68
69 • SELECT
70     p.product_name,
71     COUNT(*) AS order_count
72 FROM goldusers_signup gu
73 JOIN sales s
74     ON gu.user_id = s.user_id
75 JOIN product p
76     ON p.product_id = s.product_id
77 WHERE s.created_date >= gu.gold_signup_date
78 GROUP BY p.product_name
79 ORDER BY order_count DESC
80 LIMIT 1;
```

Q8

```
82  -- Q8. What is the total sales revenue generated in each year?
83
84 • SELECT
85     YEAR(s.created_date) AS year,
86     SUM(p.price) AS total_revenue
87 FROM sales s
88 JOIN product p
89     ON s.product_id = p.product_id
90 GROUP BY YEAR(s.created_date)
91 ORDER BY year ASC;
```

Q9

```
93  -- Q9. How has the sales revenue changed over the years?
94
95  • SELECT
96      year,
97      total_revenue AS current_year_revenue,
98      LAG(total_revenue) OVER(ORDER BY year) AS previous_year_revenue,
99      ROUND(
100          (total_revenue - LAG(total_revenue) OVER(ORDER BY year)) * 100.0
101          / LAG(total_revenue) OVER(ORDER BY year),
102          2) AS percentage_change
103
104  FROM (
105      SELECT
106          YEAR(s.created_date) AS year,
107          SUM(p.price) AS total_revenue
108      FROM sales s
109      JOIN product p
110          ON s.product_id = p.product_id
111      GROUP BY YEAR(s.created_date)
112  ) t;
```

Q10

```
136  -- Q10. What is the average Gold-signup compare to just sign up for the users?
137
138 • SELECT
139     ROUND(
140         COUNT(DISTINCT g.user_id) * 100.0 / COUNT(DISTINCT u.user_id),
141         2) AS gold_user_percentage
142 FROM users u
143 LEFT JOIN goldusers_signup g
144 ON u.user_id = g.user_id;
```

Q11

```
146  -- Q11. How many gold members users have order how many numbers of time ?
147
148 • SELECT
149     g.user_id AS gold_user_id,
150     COUNT(*) AS order_count
151 FROM goldusers_signup g JOIN sales s
152 ON g.user_id = s.user_id
153 WHERE created_date >= gold_signup_date
154 GROUP BY g.user_id;
```


Q12

-- Q12. What is the total amount each customer spend on Online Food Delivery?

- ```
SELECT
 s.user_id,
 SUM(p.price) AS total_spent
FROM sales s LEFT JOIN product p
ON s.product_id = p.product_id
GROUP BY s.user_id
ORDER BY total_spent DESC;
```

## Q13

-- Q13. What is the frequency of customer visits to the online platform?

```
SELECT
 user_id,
 COUNT(DISTINCT created_date) AS visit_frequency
FROM sales
GROUP BY user_id
ORDER BY visit_frequency DESC;
```

## Q14

-- Q14. What was the first order purchase by each customer ?

```
WITH rankcte as (
 SELECT
 user_id,
 product_name,
 row_number() OVER(PARTITION BY user_id ORDER BY created_date) AS rnk
 FROM sales s LEFT JOIN product p
 ON s.product_id = p.product_id
)

SELECT user_id, product_name FROM rankcte
WHERE rnk = 1;
```

## Q15

-- Q15. What is the most purchase item on the menu and how many times was it purchased by all customers?

```
SELECT
 s.product_id,
 product_name,
 COUNT(created_date) AS order_count
FROM sales s LEFT JOIN product p
ON s.product_id = p.product_id
GROUP BY product_id, product_name
ORDER BY order_count DESC
LIMIT 1;
```

## Q16

```
-- Q16. Which item was most popular for each customer ?
WITH product_count AS
(
 SELECT
 s.user_id,
 p.product_name,
 COUNT(*) AS order_count
 FROM sales s
 JOIN product p
 ON s.product_id = p.product_id
 GROUP BY s.user_id, p.product_name
)
ranked_products AS
(
 SELECT
 *,
 row_number() OVER(PARTITION BY user_id ORDER BY order_count DESC) AS rnk
 FROM product_count
)

SELECT user_id, product_name, order_count FROM ranked_products WHERE rnk = 1;
```

## Q17

-- Q17. Which item was purchase first by the customer after they become a member ?

WITH cte AS

```
(
SELECT
 g.user_id,
 created_date,
 p.product_id,
 p.product_name
FROM goldusers_signup g JOIN sales s
ON g.user_id = s.user_id
LEFT JOIN product p
ON p.product_id = s.product_id
WHERE created_date >= gold_signup_date
),
```

cte2 AS

```
(
SELECT
 *,
 row_number() OVER(PARTITION BY user_id ORDER BY created_date) AS rnk
FROM cte
)
```

SELECT \* FROM cte2

WHERE rnk = 1;

## Q18

-- Q18.Which item was purchase before the customer become a member ?

```
WITH cte AS
(
 SELECT
 g.user_id,
 created_date,
 p.product_id,
 p.product_name
 FROM goldusers_signup g JOIN sales s
 ON g.user_id = s.user_id
 LEFT JOIN product p
 ON p.product_id = s.product_id
 WHERE created_date < gold_signup_date
),

cte2 AS
(
 SELECT *,
 ROW_NUMBER() OVER(PARTITION BY user_id ORDER BY created_date DESC) AS rnk
 FROM cte
)

SELECT * FROM cte2
WHERE rnk = 1;
```

## Q19

-- Q19. What is the total orders and amount spent for each member before they become a member ?

```
SELECT
 g.user_id,
 COUNT(created_date) AS order_count,
 SUM(price) AS amount_spent
FROM goldusers_signup g LEFT JOIN sales s
ON g.user_id = s.user_id
LEFT JOIN product p ON p.product_id = s.product_id
WHERE created_date < gold_signup_date
GROUP BY g.user_id
ORDER BY amount_spent DESC;
```



## Q20

-- Q20. Rank all the transactions for each member whenever they are a XYZ gold member for every non gold member Transaction marks as na ?

```
SELECT
 user_id,
 created_date,
 product_id,

 CASE
 WHEN transaction_rank IS NULL THEN 'NA'
 ELSE CAST(transaction_rank AS CHAR)
 END AS transaction_rank

FROM (
 SELECT
 s.user_id,
 s.created_date,
 s.product_id,

 CASE
 WHEN s.created_date >= g.gold_signup_date
 THEN ROW_NUMBER() OVER (
 PARTITION BY s.user_id
 ORDER BY s.created_date
)
 END AS transaction_rank

 FROM sales s
 LEFT JOIN goldusers_signup g
 ON s.user_id = g.user_id
) t;
```

# Final Insights & Conclusion

## Key Insights

- ✓ Gold users spend more
- ✓ Engagement increases after membership
- ✓ Few products drive majority revenue
- ✓ Personalized behavior patterns exist

## Recommendations

- ✓ Promote gold membership
- ✓ Target high-value users
- ✓ Recommend products based on behavior

**Thank you.**